

Sequential data

- ① Time series
 - ▶ Financial data analysis: stock market, commodities, Forex
 - ▶ Healthcare: pulse rate, sugar level (from medical equipment and wearables)
- ② Text and speech: speech understanding, text generation
- ③ Spatiotemporal data
 - ▶ Self-driving and object tracking
 - ▶ Plate tectonic activity
- ④ Physics: jet identification
- ⑤ etc.

Sequence modeling 0: Language model

Goals:

- Estimate the probability that a given sequence of words occurs in a specific language.
- Model the most probable next word for a given sequence of words.

Jurafsky & Martin (2015), chapter 4, and Chen & Goodman (1999)

Examples from Jurafsky & Martin (2015)

- (1) Please turn your homework ...
What is a probable continuation? Rather *in* or *over* and not *refrigerator*.
- (2) a. all of a sudden I notice three guys standing on the sidewalk
b. on guys all I of notice sidewalk three a sudden standing the
Which of the two word orders is better?

Language model (LM): Probabilistic model that gives $P(w_1 \dots w_n)$ and $P(w_n | w_1 \dots w_{n-1})$

Evaluating language models

The data is usually separated into

- a training set (80% of the data),
- a test set (10% of the data),
- and sometimes a development set (10% of the data).

The model is estimated from the training set. The higher the probability of the test set, the better the model.

Instead of measuring the probability of the test set, LMs are usually evaluated with respect to the *perplexity* of the test set. The higher the probability, the lower the perplexity.

Evaluating language models

The perplexity of a test set $W = w_1 w_2 \dots w_N$ is defined as

$$\begin{aligned} PP(W) &= P(W)^{-\frac{1}{N}} \\ &= \sqrt[N]{\frac{1}{P(W)}} \\ &= \sqrt[N]{\frac{1}{P(w_1 w_2 \dots w_N)}} \\ &= \sqrt[N]{\frac{1}{\prod_{k=1}^N P(w_k | w_1^{k-1})}} \end{aligned}$$

A different way to think about perplexity: it measures the *weighted average branching factor* of a language.

Example

$L = \{a, b, c, d\}^*$. Frequencies are such that $P(a) = P(b) = P(c) = P(d) = \frac{1}{4}$ (independent from the context).

For any $w \in L$, given this model, we obtain

$$PP(w) = \sqrt[|w|]{\frac{1}{\prod_{k=1}^{|w|} \frac{1}{4}}} = \sqrt[|w|]{\frac{1}{\frac{1}{4}^{|w|}}} = \sqrt[|w|]{4^{|w|}} = 4$$

The perplexity of any $w \in L$ under this model is 4.

Example

$L = \{a, b, c, d\}^*$. Words in the language contain three times as many *as* as they contain *bs*, *cs* or *ds*. $P(a) = \frac{1}{2}$ and $P(b) = P(c) = P(d) = \frac{1}{6}$. For any $w \in L$ with these frequencies and with $|w| = 6n$:

$$PP(w) = \sqrt[6n]{\frac{1}{\prod_{k=1}^n \frac{1}{2 \cdot 2 \cdot 2 \cdot 6 \cdot 6 \cdot 6}}} = \sqrt[6n]{2^{6n} \cdot \sqrt{3}^{6n}} = 2\sqrt{3} = 3.46$$

Assume that we use the same model but test it on a W with equal numbers of *as*, *bs*, *cs* and *ds*, $|W| = 4n$. Then we get

$$PP(W) = \sqrt[4n]{\frac{1}{\prod_{k=1}^n \frac{1}{2 \cdot 6 \cdot 6 \cdot 6}}} = \sqrt[4n]{2^{4n} \cdot 3^{3n}} = 2 \sqrt[4n]{3^{\frac{3}{4}4n}} = 2\sqrt[4]{27} = 4.56$$

Sequence modelling I

Sequence classification

- ① $\mathbf{x} = x_1, x_2, \dots, x_n, x_i \in V$ - objects
- ② $y \in \{1, \dots, L\}$ - labels
- ③ $\{(\mathbf{x}^{(1)}, y_1), (\mathbf{x}^{(2)}, y_2), \dots, (\mathbf{x}^{(m)}, y_m)\}$ - training data

Classification problem: $\gamma : \mathbf{x} \rightarrow y$

- ① Activity recognition: x - pulse rate, y - activity (walking, running, peace)
- ② Opinion mining: x - sentence, y - sentiment (positive, negative)
- ③ Trading: x - stock market, y - action (sell, buy, do nothing)

Sequence modelling II

Sequence labelling

- ① $\mathbf{x} = x_1, x_2, \dots, x_n, x_i \in V$ - objects
- ② $\mathbf{y} = y_1, y_2, \dots, y_n, y_i \in \{1, \dots, L\}$ - labels
- ③ $\{(\mathbf{x}^{(1)}, \mathbf{y}^{(1)}), (\mathbf{x}^{(2)}, \mathbf{y}^{(2)}), \dots, (\mathbf{x}^{(m)}, \mathbf{y}^{(m)})\}$ - training data
- ④ exponential number of possible solutions : if $\text{length}(\mathbf{x}) = n$, there are L^n possible solutions

Classification problem: $\gamma : \mathbf{x} \rightarrow \mathbf{y}$

- ① Part of speech tagging: x - word, y - part of speech (verb, noun, etc.)
- ② Genome annotation: x - DNA, y - genes
- ③ HEP tracking: x - a set of hits with backgrounds, y - hit classification

Sequence labelling tasks

POS tagging and Named Entity Recognition

X (words)	the	cat	sat	on	a	mat
Y (tags)	DET	NOUN	VERB	PREP	DET	NOUN

Table: POS tagging

Alex	is	going	to	Los	Angeles
B-PER	O	O	O	B-LOC	I-LOC

Table: NER (IOB2)

Alex	travels	with	Marty	A.	Rick	to	NY	city
S-PER	O	O	B-PER	I-PER	E-PER	O	B-LOC	E-LOC

Table: NER (IOBES)

Sequence modelling III

Sequence transduction / transformation

- ① $\mathbf{x} = x_1, x_2, \dots, x_n, x_i \in V_{source}$ - objects
- ② $\mathbf{y} = y_1, y_2, \dots, y_n, y_i \in V_{target}$ - objects
- ③ $\{(\mathbf{x}^{(1)}, \mathbf{y}^{(1)}), (\mathbf{x}^{(2)}, \mathbf{y}^{(2)}), \dots, (\mathbf{x}^{(m)}, \mathbf{y}^{(m)})\}$ - training data
- ④ $\mathbf{x}^{(1)}, \mathbf{y}^{(1)}$ are of different length

Transduction problem: $\mathbf{x}_{source} \rightarrow \mathbf{y}_{target}$

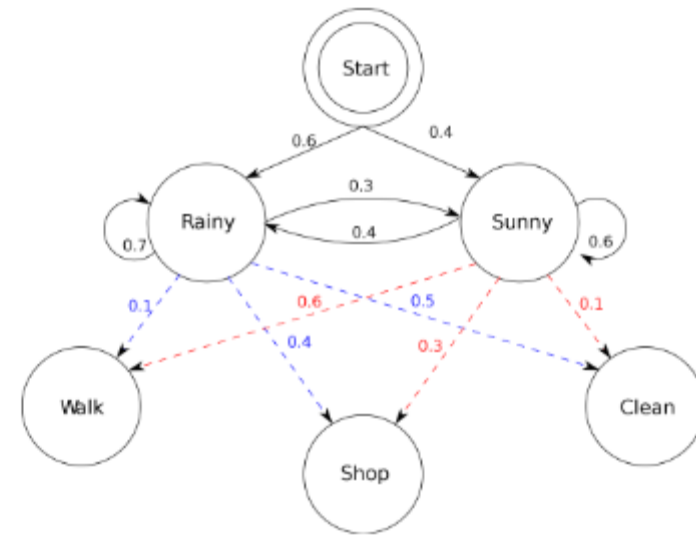
- ① Machine translation: x - sentence in German, y - sentence in English
- ② Speech recognition: x - spoken language, y - text
- ③ Chat bots: x - question, y - answer

Traditional ML approaches to sequence modeling

- Hidden Markov Models (HMM)
- Conditional Random Fields (CRF)
- Local classifier: for each x define features, based on x_{-1} , x_{+1} , etc, and perform classification n times

Problems:

- 1 Markov assumption: fixed length history
- 2 Computation complexity

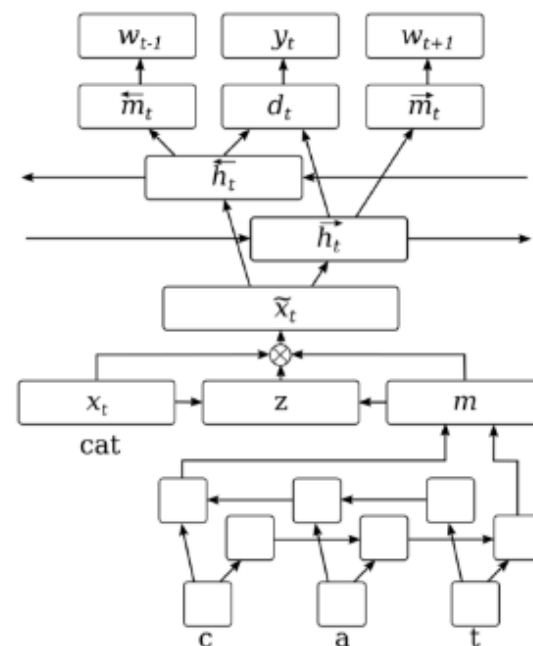


DL approaches to sequence modeling

- Neural networks
- Recurrent neural network and its modifications: LSTM, GRU, Highway
- 2D Convolutional Neural Network
- Transformer
- Pointer network

Problems:

- 1 Training time
- 2 Amount of training data



Neural language model

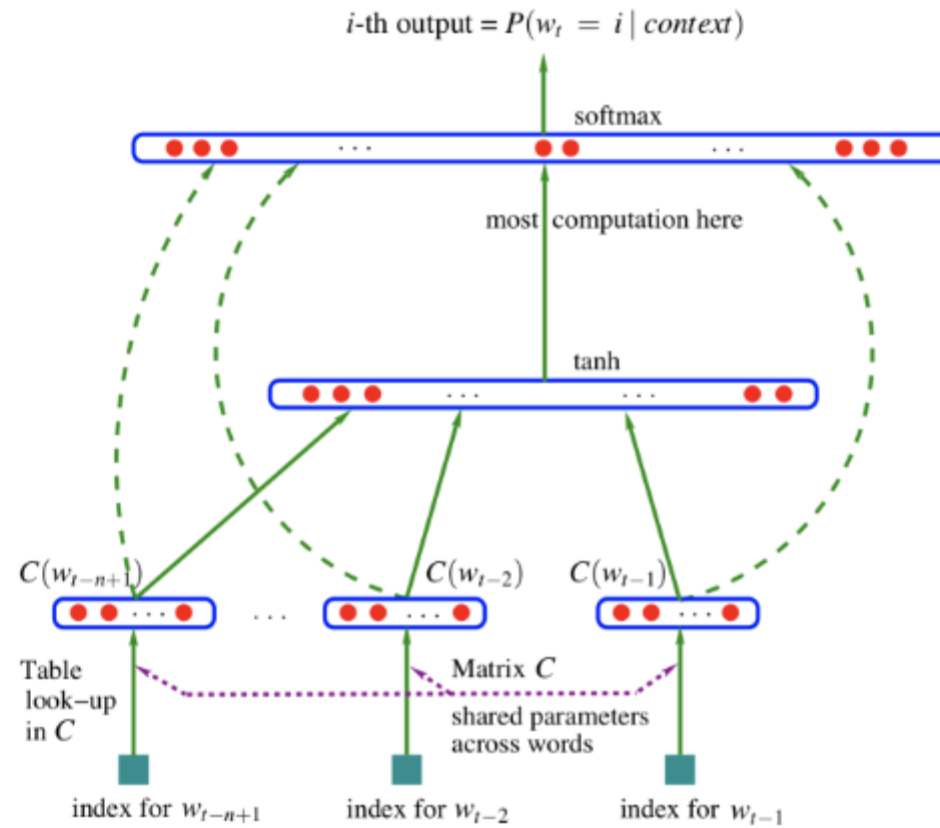
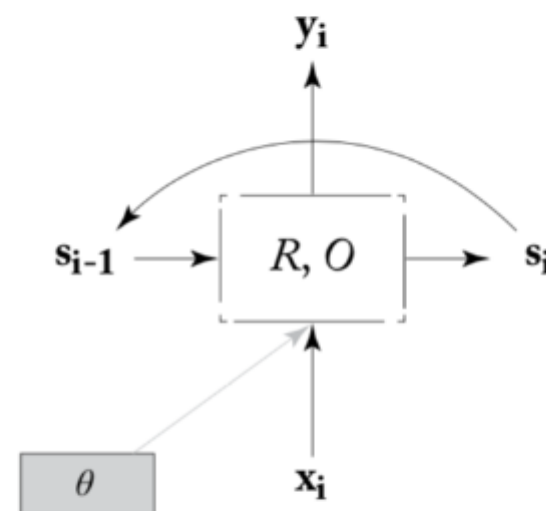


Figure: Neural language model

Recurrent neural network

- Input: sequence of vectors
- $x_{1:n} = x_1, x_2, \dots, x_n, x_i \in \mathbb{R}^{d_{in}}$
- Output: a single vector
 $y_n = RNN(x_{1:n}), y_n \in \mathbb{R}^{d_{out}}$
- For each prefix $x_{1:j}$ define an output vector y_j :
 $y_j = RNN(x_{1:j})$
- RNN^* is a function returning this sequence for input sequence $x_{1:n}$:
 $y_{1:n} = RNN^*(x_{1:n}), y_i \in \mathbb{R}^{d_{out}}$



Sequence modelling with RNN

① Sequence classification

Put a dense layer on top of RNN to predict the desired class of the sequence after the whole sequence is processed

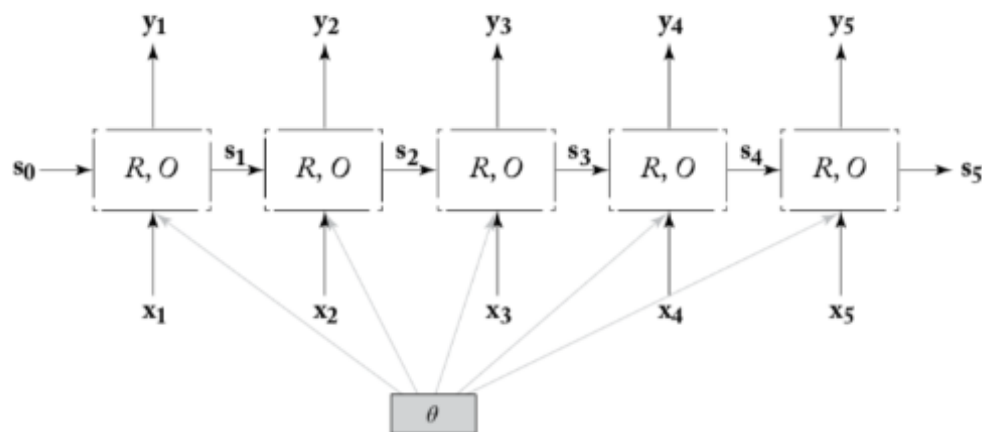
$$p(l_j | \mathbf{x}_{1:n}) = \text{softmax}(RNN(\mathbf{x}_{1:n}) \times W + b)_{[j]}$$

② Sequence labelling

Produce an output y_i for each input RNN reads in. Put a dense layer on top of each output to predict the desired class of the input

$$p(l_j | \mathbf{x}_j) = \text{softmax}(RNN(\mathbf{x}_{1:j}) \times W + b)_{[j]}$$

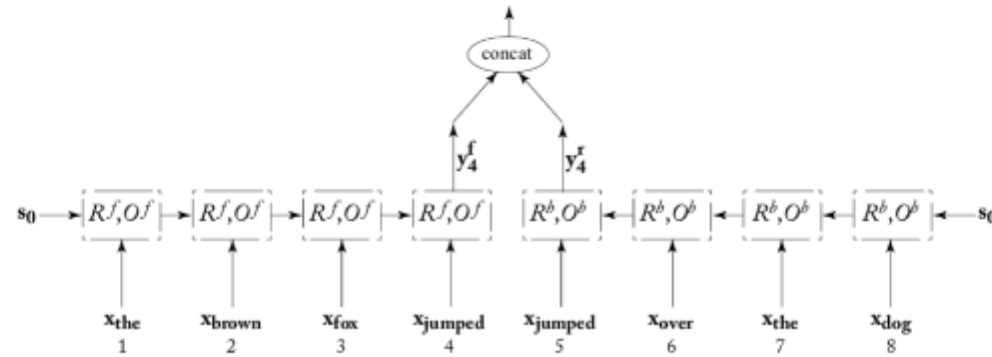
RNN unrolled



$$\begin{aligned} s_4 &= R(s_3, x_4) = R(R(s_2, x_3), x_4) = R(R(R(s_1, x_2), x_3), x_4) = \\ &= R(R(R(R(s_0, x_1), x_2), x_3), x_4) \end{aligned}$$

Bidirectional RNN (Bi-RNN)

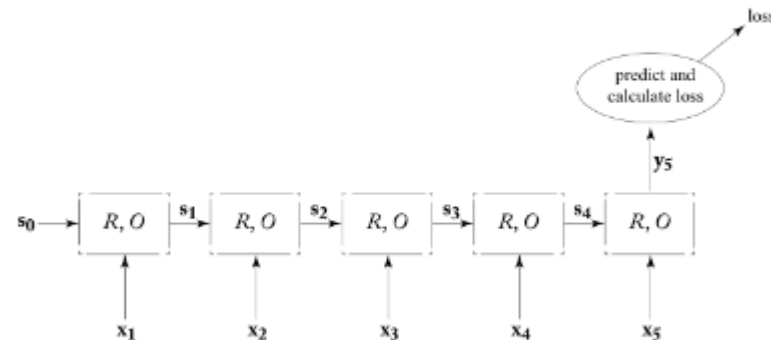
The input sequence can be read from left to right and from right to left.
Which direction is better?



$$biRNN(x_{1:n}, i) = y_i = [RNN^f(x_{1:i}); RNN^r(x_{n:i})]$$

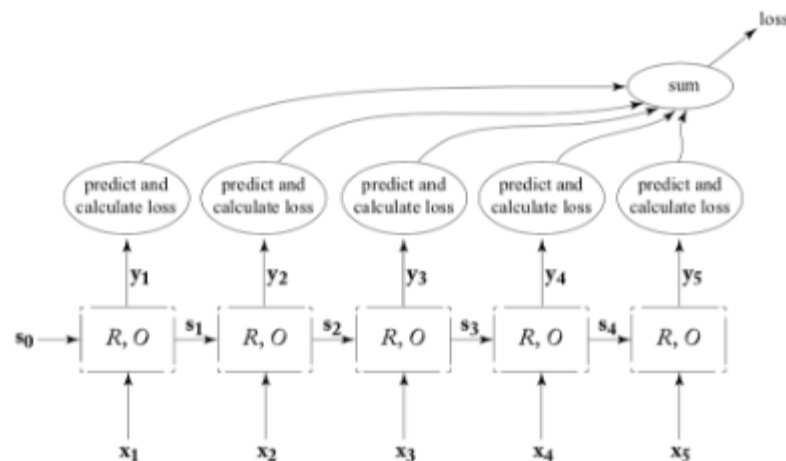
Sequence classification

- $\hat{y}_n = O(s_n)$
- $\text{prediction} = \text{MLP}(\hat{y}_n)$
- Loss: $L(\hat{y}_n, y_n)$
- L can take any form: cross entropy, hinge, margin, etc.

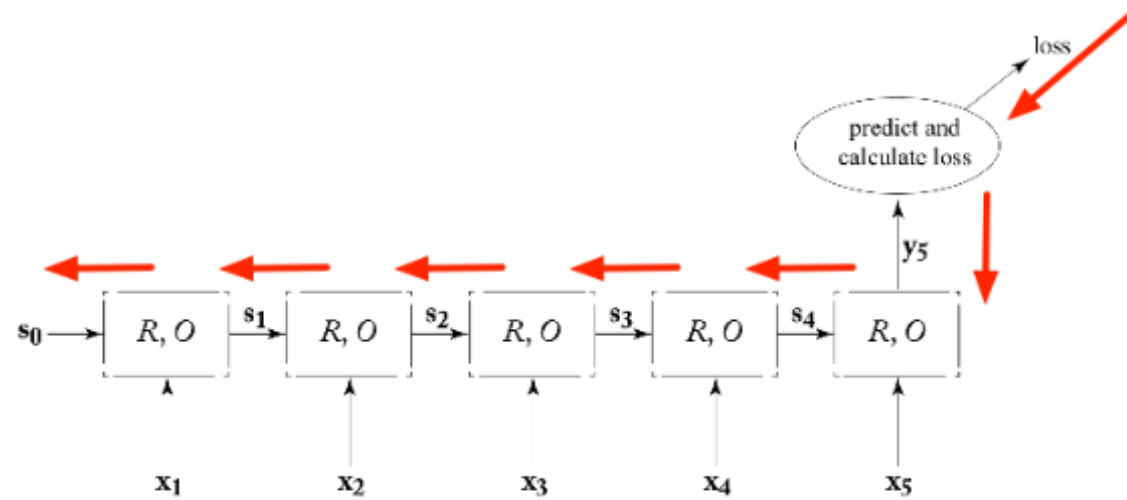


Sequence labelling

- Output $\hat{t}_i$ for each input $x_{1,i}$
- Local loss: $L_{local}(\hat{t}_i, t_i)$
- Global loss:
$$L(\hat{t}_n, t_n) = \sum_i L_{local}(\hat{t}_i, t_i)$$
- L can take any form: cross entropy, hinge, margin, etc.



Backpropagation through time



$$s_i = R(x_i, s_{i-1}) = g(s_{i-1}W^s + x_iW^x + b)$$

$$\text{Chain rule: } \frac{\partial L}{\partial w} = \frac{\partial L}{\partial p(\hat{y}_5)} \frac{\partial p(\hat{y}_5)}{\partial s_4} \left(\frac{\partial s_4}{\partial w} + \frac{\partial s_4}{\partial s_3} \frac{\partial s_3}{\partial w} + \frac{\partial s_4}{\partial s_3} \frac{\partial s_3}{\partial s_2} \frac{\partial s_2}{\partial w} + \dots \right)$$

Vanishing gradient problem

Chain rule: $\frac{\partial L}{\partial w} = \frac{\partial L}{\partial p(\hat{y}_5)} \frac{\partial p(\hat{y}_5)}{\partial s_4} \left(\frac{\partial s_4}{\partial w} + \frac{\partial s_4}{\partial s_3} \frac{\partial s_3}{\partial w} + \frac{\partial s_4}{\partial s_3} \frac{\partial s_3}{\partial s_2} \frac{\partial s_2}{\partial w} + \dots \right)$
 g – sigmoid

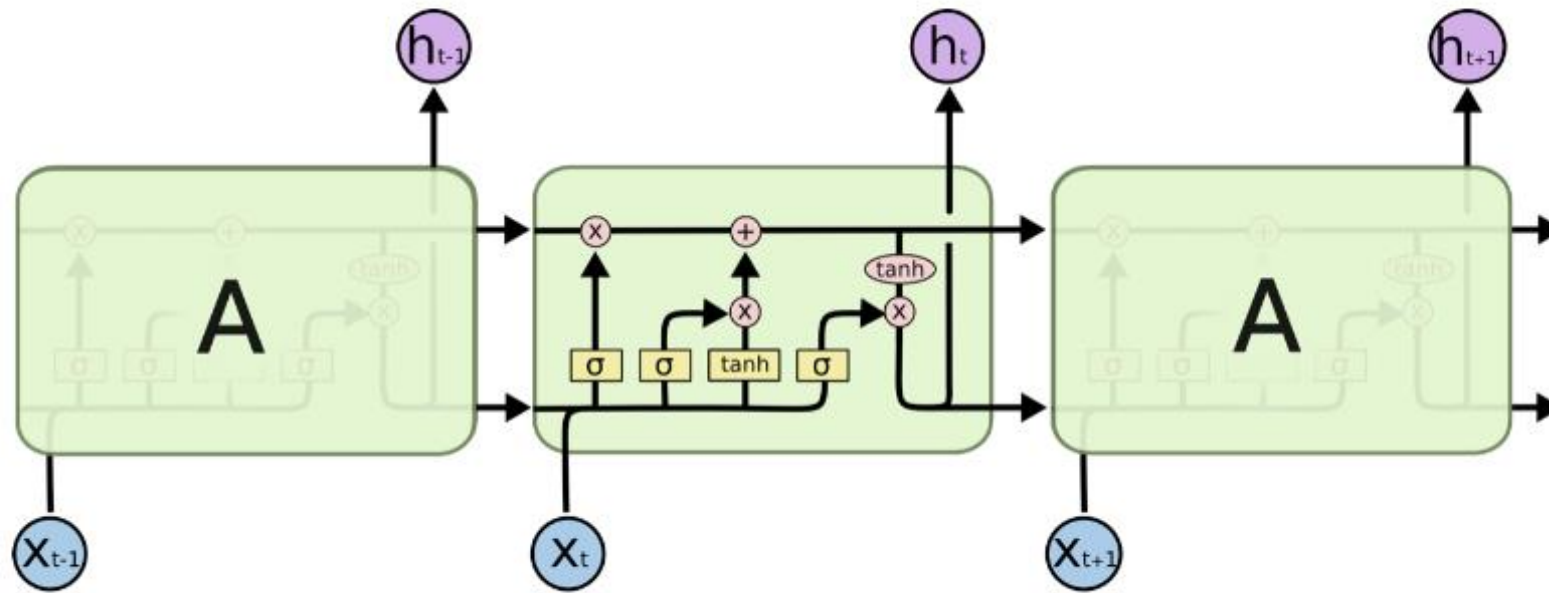
- ① Many sigmoids near 0 and 1
 - ▶ Gradients $\rightarrow 0$
 - ▶ Not training for long term dependencies
- ② Many sigmoids > 1
 - ▶ Gradients $\rightarrow +\infty$
 - ▶ Not training again

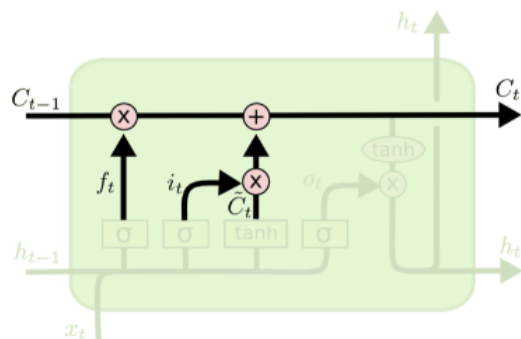
Solution: gated architectures (LSTM and GRU)

Controlled memory access

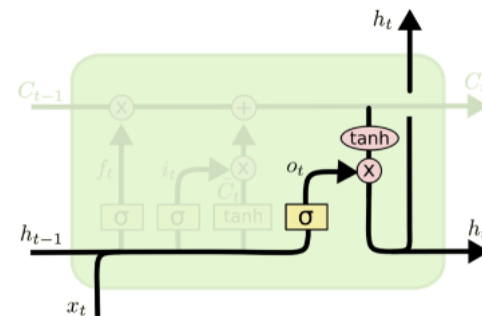
- Entire memory vector is changed: $s_{i+1} = R(x_i, s_i)$
- Controlled memory access: $s_{i+1} = g \odot R(x_i, s_i) + (1 - g)s_i$
 $g \in [0, 1]^d, s, x \in \mathbb{R}^d$
- Differential gates: $\sigma(g), g' \in \mathbb{R}^d$
- This controllable gating mechanism is the basis of the LSTM and the GRU architectures

Long short term memory



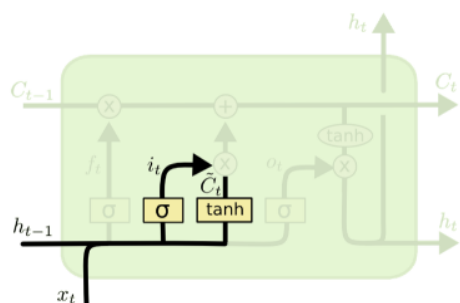


$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$



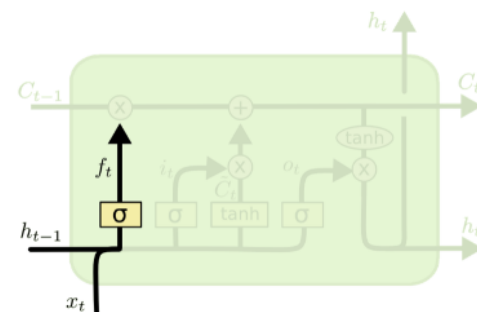
$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$



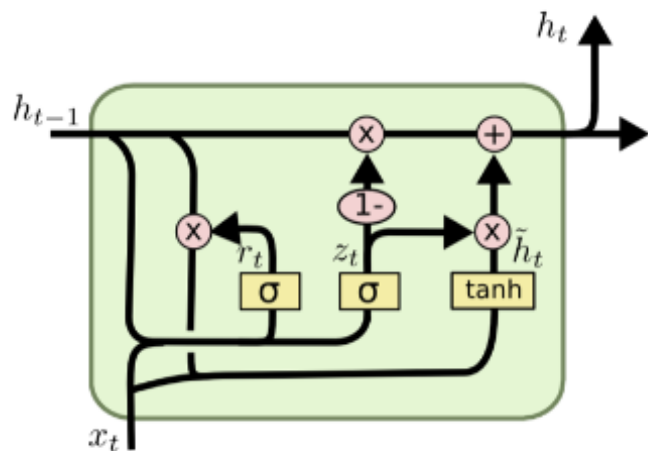
$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$



$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

Gated recurrent unit



$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

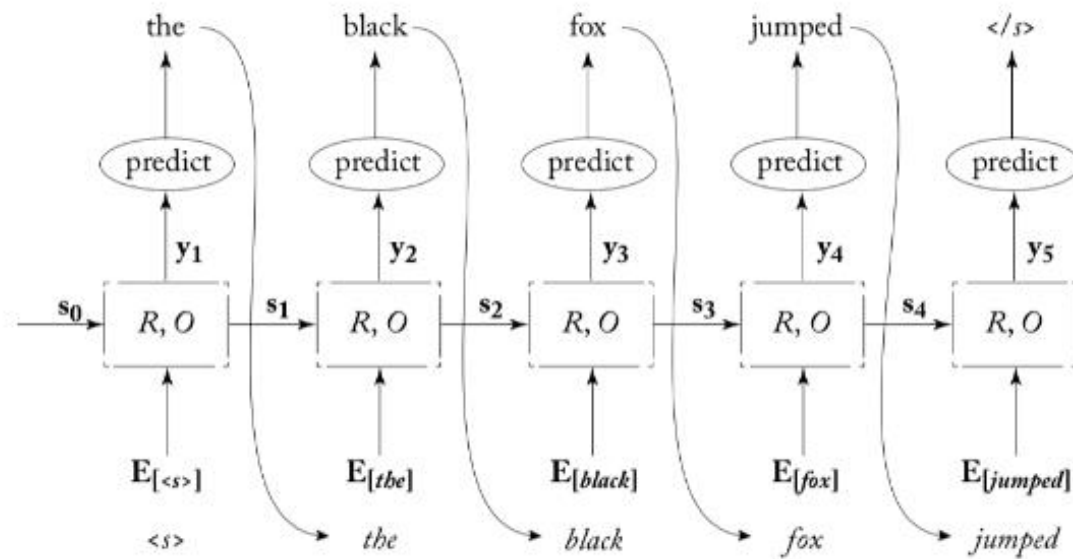
$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

Sequence generation

Teacher forcing: $x := \langle s \rangle x, y := x \langle /s \rangle$

$x : \langle s \rangle x_1 x_2 \dots x_n$

$y : x_1 x_2 \dots x_n \langle /s \rangle$



Pros and cons of RNNs

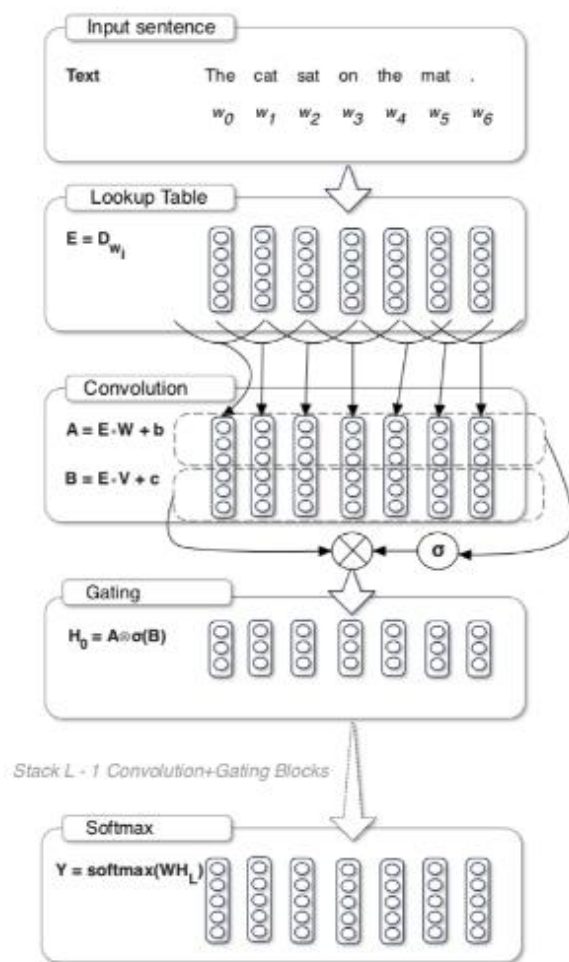
① Advantages:

- ▶ RNNs are popular and successful for variable-length sequences
- ▶ The gating models such as LSTM are suited for long-range error propagation

② Problems:

- ▶ The sequentiality prohibits parallelization within instances
- ▶ Long-range dependencies still tricky, despite gating

Language Modeling with Gated Convolutional Networks



- **Embeddings** $\in D^{|V| \times e}$
- **Input:** $w_0, \dots, w_n \rightarrow E = [D_{w_0}, \dots, D_{w_n}]$
- **Hidden layers:** $h_0, \dots, h_n$:

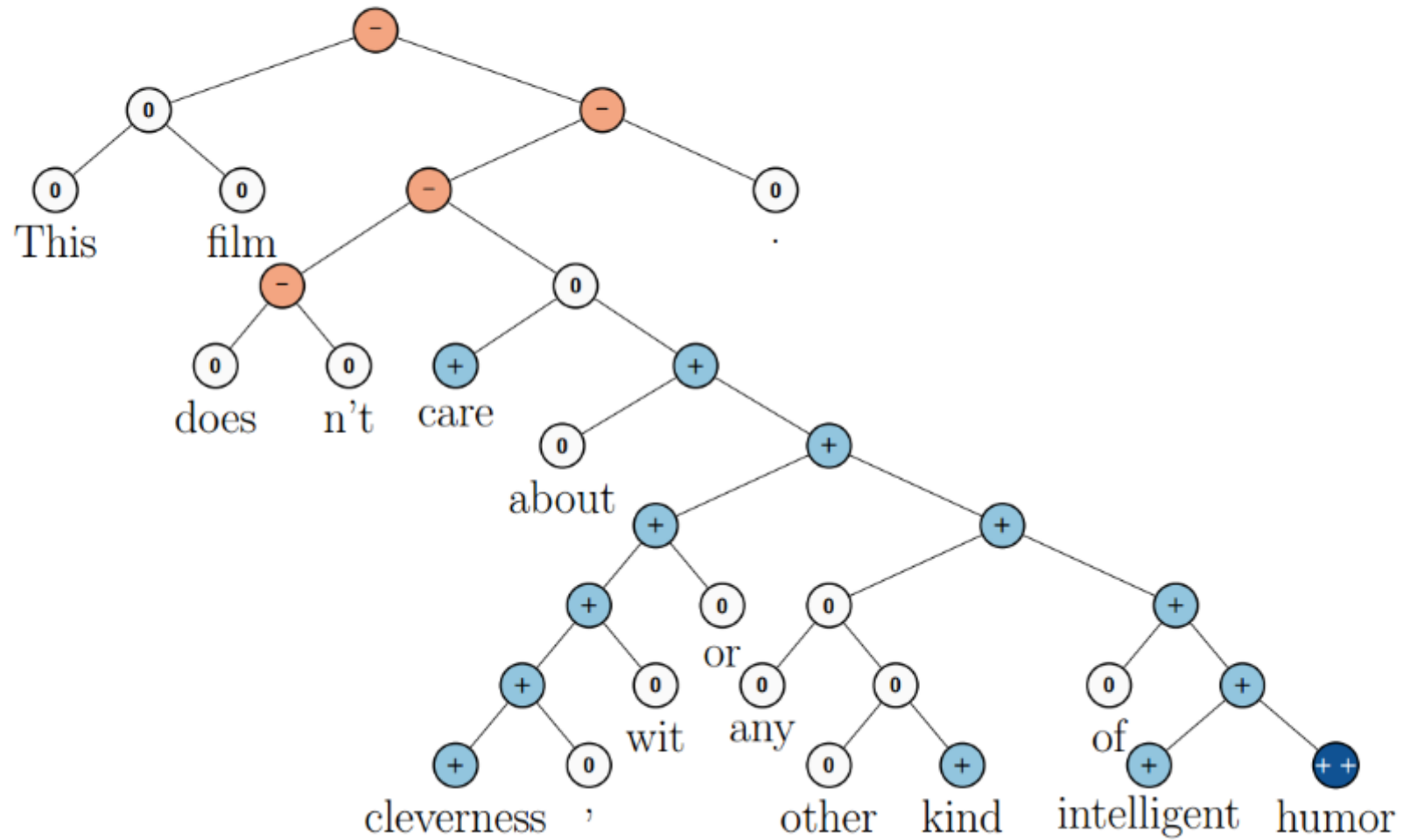
$$h_l(X) = (X \times W + b) \circ \sigma(X \times V + c)$$

- **Gated linear unit:** $X \circ \sigma(X)$
- **Output:** $Y = \text{softmax}(WH_L)$

Modeling trees with Recursive NN

- Input: $x_1, x_2, \dots, x_n$
- A binary tree T can be represented as a unique set of triplets (i, k, j) , s.t. $i < k < j$, $x_{i:j}$ is parent of $x_{i:k}, x_{k+1:j}$
- RecNN takes as an input a binary tree and returns as output a corresponding set of inside state vectors $\mathbf{s}_{i:j}^A \in \mathbb{R}^d$
- Each state vector $\mathbf{s}_{i:j}^A$ represents the corresponding tree node $q_{i:j}^A$ and encodes the entire structure rooted at that node

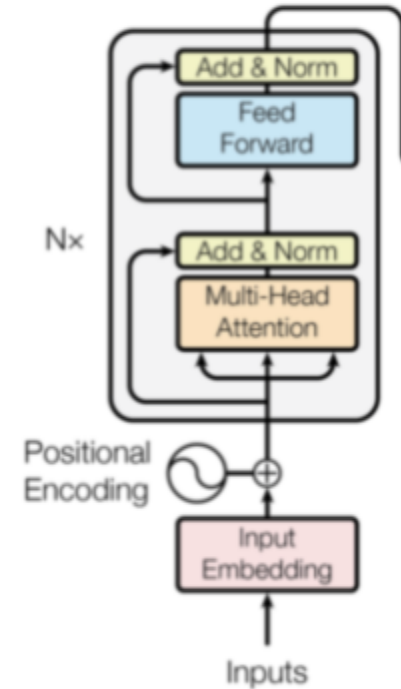
RecNN



The Transformer

An alternative architecture to RNN which allows of parallel and faster training

- Several layers of identical modules
- Each module consists of Multi-Head Attention and Feed Forward layers
- Input: embeddings. To get embeddings for numerical input, apply any dense layer
- Positional embeddings to make use of the order of the sequence

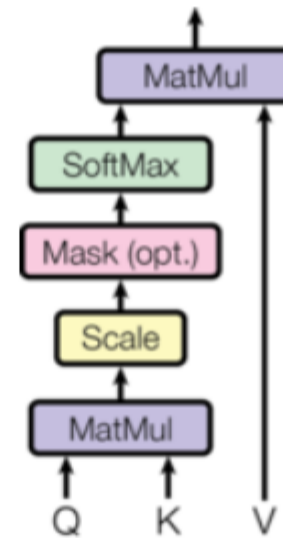


Scaled Dot-Product Attention

An attention function can be described as mapping a query and a set of key-value pairs to an output, where the query, keys, values, and output are all vectors:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V,$$

where the input consists of queries Q and keys K of dimension d_k and values V of dimension d_v



Modern RNNs: RetNet

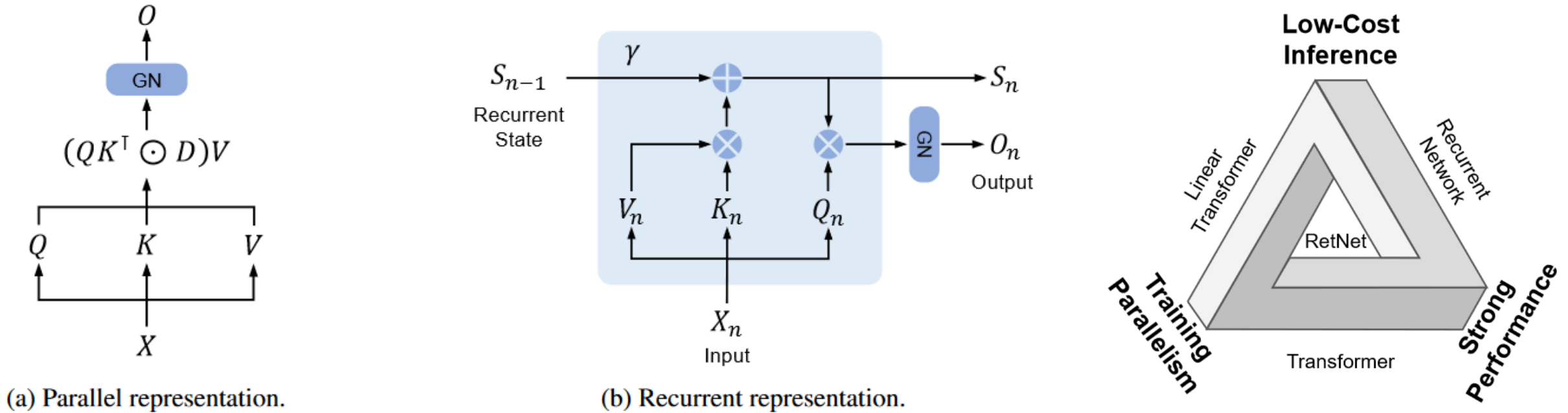


Figure 3: Dual form of RetNet. “GN” is short for GroupNorm.

The **RetNet model** (short for **Retention Network**, introduced by Microsoft Research in 2023) is a new sequence modeling architecture proposed as an alternative to Transformers.

Its main idea is to keep the parallelizability of Transformers while introducing recurrent-like efficiency for very long sequences.

Modern RNNs: RetNet

1. Retention Mechanism

1. RetNet replaces the Transformer's self-attention with a new operator called **retention**.
2. It computes token representations with a **decay-weighted recurrence**, which combines elements of attention (parallelizable) and RNNs (sequential memory).
3. This allows it to **remember long-term dependencies** without quadratic complexity.

2. Linear Time & Memory Complexity

1. Like linear-attention methods, RetNet has **$O(n)$** complexity in sequence length (vs. $O(n^2)$ for standard Transformers).
2. Scales well for very long contexts (tens of thousands of tokens).

3. Parallel + Recurrent Modes

1. RetNet can run in **parallel mode** (fast training, like Transformers) and **recurrent mode** (efficient autoregressive inference, like RNNs).

Modern RNNs: Mamba

1. State Space Model (SSM) Backbone

- Mamba is based on **selective state space models (SSMs)** rather than attention.
- SSMs update a hidden state with each token, similar to RNNs, but with a **structured linear dynamical system** that allows efficient sequence modeling.

2. Selectivity Mechanism

- Introduces a **data-dependent gating/selectivity** mechanism, letting the model decide which past information to keep or forget.
- This makes memory usage **adaptive** rather than uniform (unlike RetNet's exponential decay).

3. Linear Time Complexity

- Like RetNet, Mamba achieves **$O(n)$** complexity in sequence length.
- Memory and compute scale linearly, making it suitable for **very long contexts (hundreds of thousands of tokens)**.

4. Hardware Efficiency

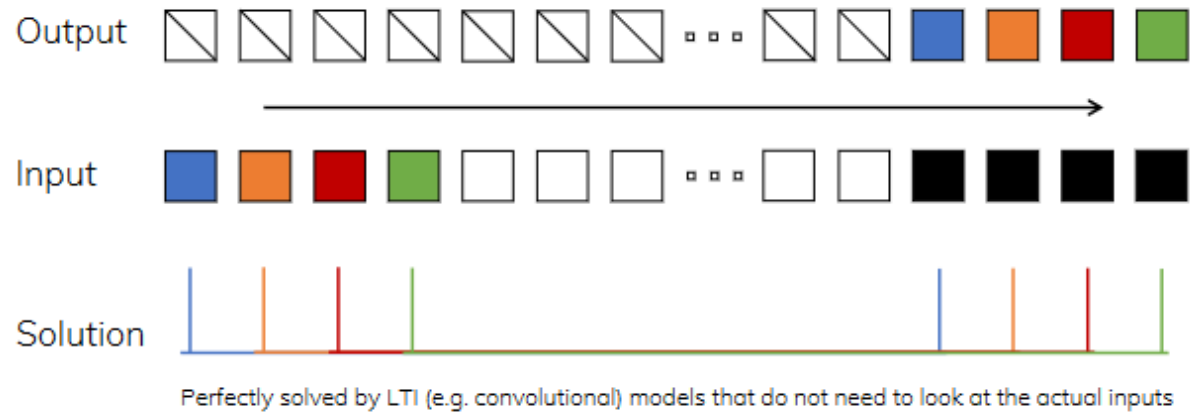
- The architecture is designed to map efficiently to GPUs/TPUs.
- Mamba is **much faster than Transformers** for long sequences, both in training and inference.

5. Continuous State Representation

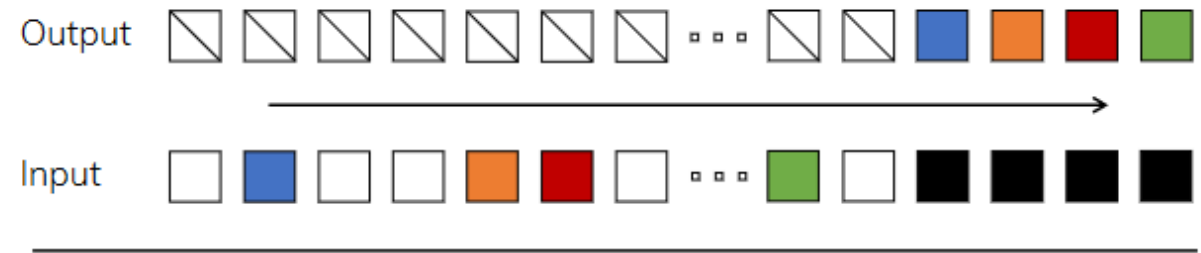
- Maintains a **continuous hidden state** (like an RNN) that evolves over time.
- This allows **constant-time per-token inference**.

Modern RNNs : Mamba

Copying



Selective Copying



Induction Heads

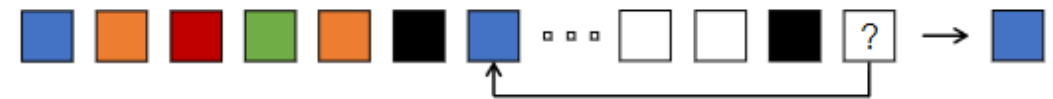
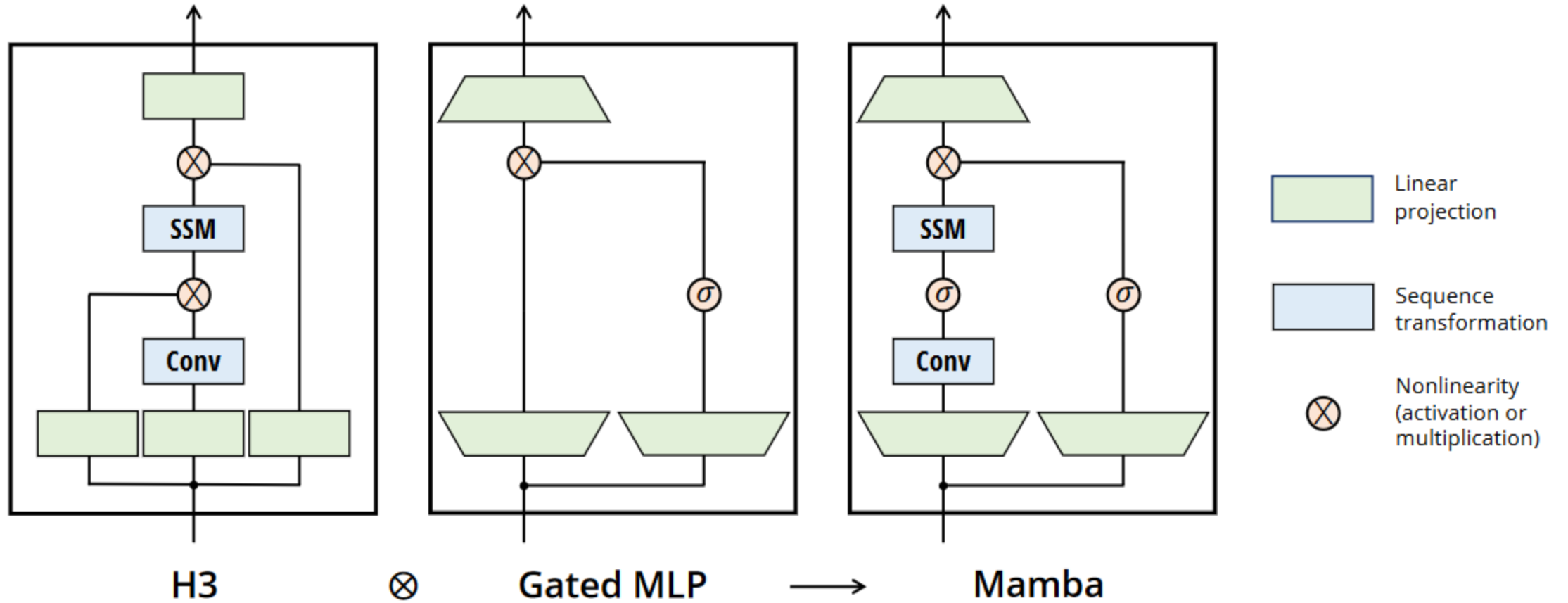
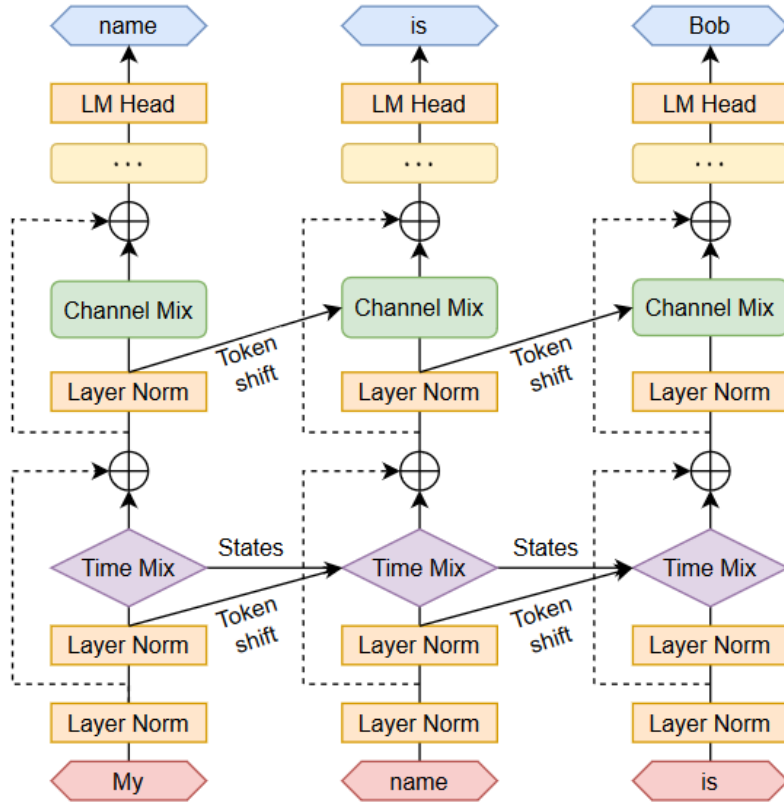


Figure 2: (Left) The standard version of the Copying task involves constant spacing between input and output elements and is easily solved by time-invariant models such as linear recurrences and global convolutions. (Right Top) The Selective Copying task has random spacing in between inputs and requires time-varying models that can *selectively* remember or ignore inputs depending on their content. (Right Bottom) The Induction Heads task is an example of associative recall that requires retrieving an answer based on context, a key ability for LLMs.

Modern RNNs: Mamba



Modern RNNs: RWKV



- R : The **Receptance** vector acts as the receiver of past information.
- W : The **Weight** signifies the positional weight decay vector, a trainable parameter within the model.
- K : The **Key** vector performs a role analogous to K in traditional attention mechanisms.
- V : The **Value** vector functions similarly to V in conventional attention processes.

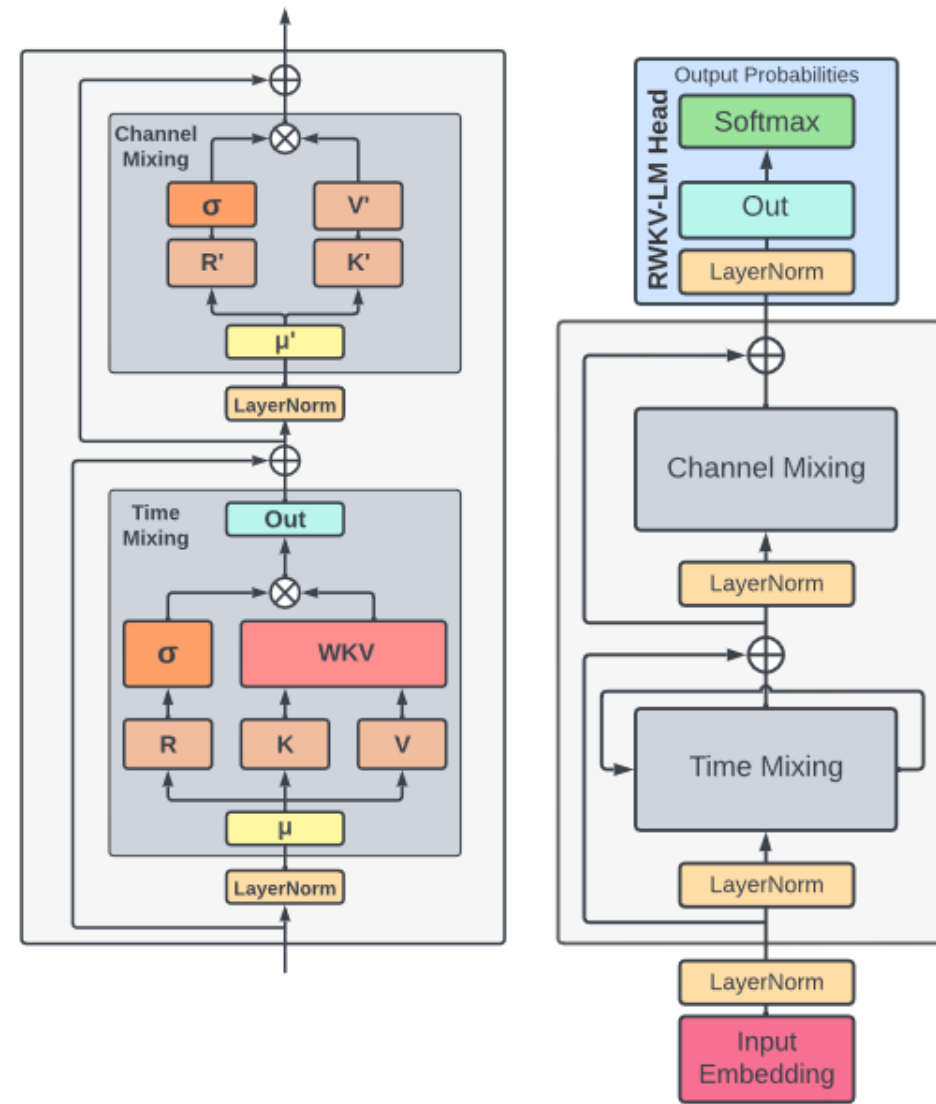


Figure 2: Elements within an RWKV block (left) and the complete RWKV residual block, equipped with a final head for language modeling (right).

Modern RNNs: RWKV

1. RNN-Transformer Hybrid

1. RWKV is built as a **recurrent neural network** but trained with **Transformer-like weight sharing and architecture patterns**.
2. It can process sequences one token at a time (like an RNN) while keeping some of the expressivity of attention models.

2. Linear Complexity

1. Like RetNet and Mamba, RWKV has **$O(n)$** time and memory complexity.
2. It avoids quadratic attention costs, making it efficient for **very long sequences**.

3. Time-Mixing & Channel-Mixing Blocks

1. Instead of self-attention, RWKV uses **time-mixing** (temporal recurrence across tokens) and **channel-mixing** (feedforward across hidden dimensions).
2. This plays a role similar to attention + MLP in Transformers, but in a recurrent-friendly way.

4. Recurrent Inference Mode

1. RWKV can run **autoregressively like an RNN**: constant memory per token, only needing the hidden state.
2. This makes inference **highly efficient** for deployment on CPUs and edge devices.

5. Transformer-Like Training

Modern RNNs: ARMT

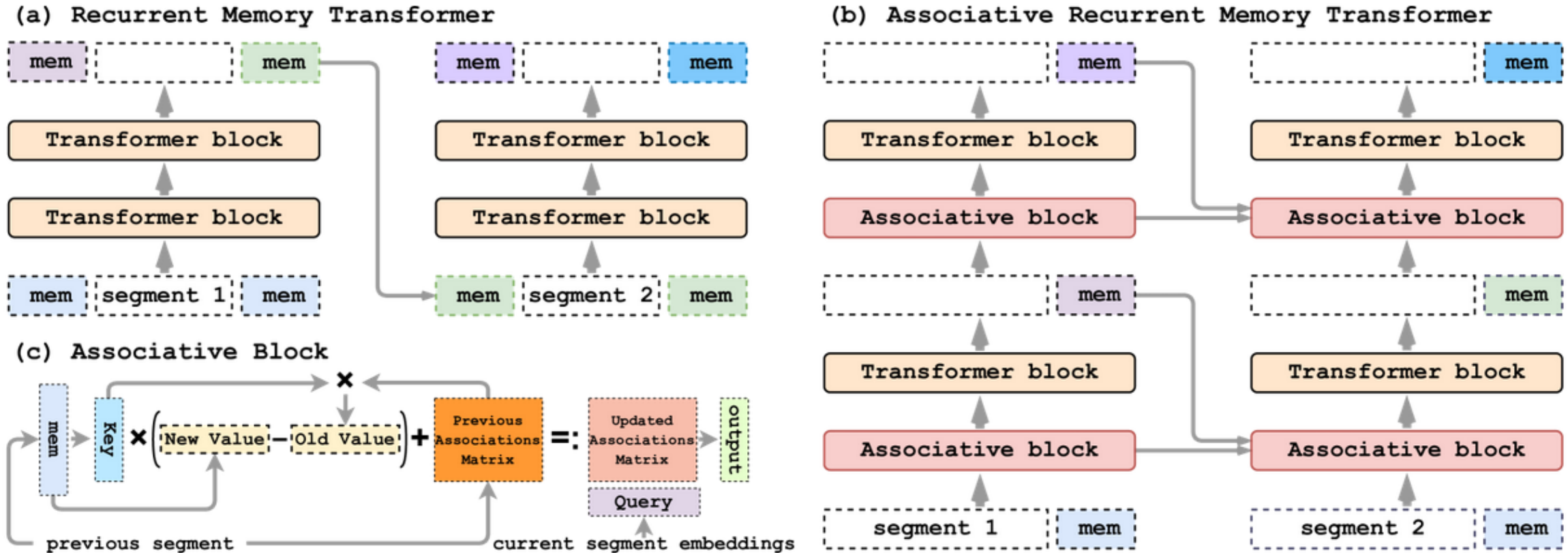


Figure 1: **ARMT augments the transformer's layers with associative memory.** (a) RMT architecture. (b) ARMT adds associative memory processing to each layer. (c) Associative memory is updated with layerwise memory representations.

Associative Recurrent Memory Transformer (ARMT) – 2024. Hybrid model combining local Transformer self-attention + **segment-level recurrence** to store and retrieve information across large contexts. Designed to handle very long sequences. “**Constant time for processing new information at each time step**” is claimed.